CS288 Assignment 3 Report

Q1 QA Data Creation

We created a dataset of 100 factoid QA pairs by navigating the eecs.berkeley.edu domain.

Two team members independently annotated a random 30% set (30 questions). We applied the required string normalization including lowercasing, and punctuation removal before calculating the agreement. We achieved an Exact Match (EM) of 80.00% and a Token-level F1 IAA of 90.18%.

Most of our conflicts originated from answer granularity. For example, one annotator extracted "2:00 pm Pacific Time" while the other chose to write "2:00 pm". To mitigate this, we use "|" as suggested to accommodate varied but correct answers (e.g., APS | American Physical Society).

Sample QA Pairs from Our Dataset:

Question	Reference Answer	URL
When did Joseph Thomas Gier become a full professor?	1958	https://eecs.berkeley.edu/about/history/gier/
How did Berkeley EECS rank in 2023 US News?	#1 | 1 | First | 1st	https://eecs.berkeley.edu/category/education/page/8/
Who is the author of the technical report "Coarse-to-fine MCMC in a seismic monitoring system"?	Xiaofei Zhou	https://www2.eecs.berkeley.edu/Pubs/TechRpts/2015/EECS-2015-252.html

Q2 Retrieval Corpus

We developed a custom web crawler based on Breadth-First Search (BFS), expanding upon the baseline code provided on Ed Discussion. To handle the extensive crawling time, we implemented a local state-saving mechanism (queue and visited logs) to ensure fault tolerance. For data deduplication, we normalized all http URLs to https to prevent redundantly fetching identical pages. During parsing, we utilized BeautifulSoup to aggressively filter out HTML noise, including <script> and <style> tags.

For text chunking, we initially experimented with an LLM-based API (deepseek api) to divide the documents contextually. However, due to severe latency and API cost constraints as the raw data volume increased, we transitioned to a sliding-window approach. We simply divided the cleaned text into fixed chunks of 250 characters, with a 50-character overlap, to maintain semantic continuity efficiently.

Our curated corpus achieved 77.68% F1 and 71.00% Exact Match (EM), significantly outperforming the official reference corpus (65.73% F1, 59.00% EM). This ~10% performance gap is primarily due to corpus scale. Our crawling extracted ~19,000 passages, compared to only ~5,000 in the reference corpus. This nearly fourfold capacity increase drastically improved recall upper bounds for niche queries.

Q3 RAG System

To meet strict hardware constraints (2 CPUs, 3GB RAM), we pre-computed BM25 and FAISS (all-MiniLM-L6-v2) indices offline. At runtime, our hybrid pipeline fetches the top 30 candidates from both indices, merging them via Reciprocal Rank Fusion (RRF). To refine results without exceeding RAM

limits, a lightweight cross-encoder (ms-marco-MiniLM-L-6-v2) reranks the top 10 RRF candidates, passing the top 8 to the LLM. For generation, we optimized Token-level F1 by using few-shot prompting to force ultra-concise answers and deploying strict regex to strip trailing punctuation and leading articles (e.g., "a", "the").

Q4. Ablations

We evaluated two key pipeline design choices:

Context Window Size (FINAL_TOP_K): Feeding the top 8 reranked passages to the LLM yielded an optimal F1 of 77.08%, outperforming the top-2 and top-4 versions. This indicates that when the reranker effectively filters noise, supplying a broader context safely prevents the truncation of crucial facts.

Cross-Encoder Reranker: Bypassing the reranker and relying solely on RRF dropped the F1 from 77.08% to 76.51%. The cross-encoder provides a layer of semantic verification, ensuring the most factually aligned passages are pushed to the top, thereby reducing LLM distraction.

Experiment	F1 Score (%)	Exact Match (%)	Total Time (s)	Latency/Q (s)
K=8 + Reranker (Optima	77.68	71.0	74.47	0.745
K=4 + Reranker	74.7	65.0	64.09	0.641
K=2 + Reranker	76.66	68.0	64.18	0.642
No Reranker (RRF Only)	77.43	70.0	36.0	0.36
BM25 Only	75.72	68.0	36.76	0.368

Q5 Error analysis

We conducted an error analysis on predictions to identify the failure modes in our RAG pipeline.

A significant portion of errors originated from reranker sub-optimality, where the cross-encoder failed to promote the goal passage to the first position although it's included in the initial hybrid retrieved set. We found that, regarding the false negatives due to metric limitations, the evaluation script penalized factually accurate responses that had minor lexical variations like numeric formatting mismatches such as "six" and "6".

To refine future evaluations, adding entity normalization and semantic similarity to the evaluation to reduce unfair penalties for minor variations.

Q6. Takeaways & Future Ideas

This project highlighted that RAG performance heavily depends on data quality and retrieval precision, rather than just LLM capability. Our primary takeaway from this project is that traditional Hybrid RAG (BM25 + Dense) merely operates in a 2-dimensional retrieval space (sparse keyword matching & dense semantic similarity). While effective for basic queries, it still fails on complex reasoning tasks due to dimension limitations. Based on this insight, for our future work (and also our final project), we propose the concept of "Manifold RAG." Instead of a simple linear combination, Manifold RAG projects documents into a high-dimensional manifold space, integrating multiple heterogeneous retrieval metrics—such as structural graph paths, document recency, and metadata features—allowing the retriever to dynamically route queries across a richer geometric representation of the corpus.

Contribution Statement

Boning Shao

Developed the RAG architecture in rag.py. Implemented hybrid retrieval using BM25 and FAISS with Reciprocal Rank Fusion (RRF) and integrated the Cross-Encoder re-ranker. Do the ablation test and compare the performance of our curated corpus and the reference corpus.

Tianyi Huang

Curated the 100-question evaluation dataset and coordinated the IAA verification. Designed the LLM prompts, including few-shot examples and negative constraints. Performed the manual error analysis on the final predictions, and authored the written report.

Yicheng Peng

Built the data pipeline, including the web scraper and HTML parser. Implemented regex-based cleaning to sanitize the retrieval corpus and generated 5,000 synthetic QA pairs for model development. Also completed the blind IAA annotation for the evaluation set.

GenAI Acknowledgement

We acknowledge the use of Google Gemini (https://gemini.google.com) to troubleshoot Python error messages and for conceptual explanations of retrieval algorithms. We used the outputs to understand the "expected string or bytes-like object" type error in our JSON text-cleaning script, debug our rag.py program, and to clarify the underlying inverted index mechanism of BM25 for our system design workflow. We have the ability to explain and independently replicate the work done in this document if asked by an instructor.